

OPERATING SYSTEMS

Unit-4: File Management

Comprehensive Study Notes

What is File Management?

File Management is the function of the operating system that manages **files** and **directories** on storage devices. It provides:

- **Organization** of data on disk
- **Access** to data for users and programs
- **Protection** of data from unauthorized access
- **Management** of storage space

File Management Functions

FILE MANAGEMENT FUNCTIONS
1. FILE CREATION & DELETION
- Create new files
- Delete existing files
- Allocate space for files
2. DIRECTORY MANAGEMENT
- Create and delete directories
- Organize files in directories
- Navigate directory structure
3. FILE OPERATIONS
- Read data from files
- Write data to files
- Seek to specific positions
- Append data to files
4. FILE PROTECTION
- Access control (read/write/execute permissions)
- User authentication
- File encryption
5. STORAGE MANAGEMENT
- Free space management
- Disk allocation
- Backup and recovery

File Attributes

Attribute	Description
-----------	-------------

Name	Human-readable identifier
Type	Data, program, system, or special file
Size	Current file size in bytes/blocks
Protection	Read/Write/Execute permissions
Time	Creation, modification, access timestamps
Location	Pointer to file data on disk

File Operations

Operation	Description
Create	Create a new file with given attributes
Open	Open an existing file for reading/writing
Read	Read data from a file at current position
Write	Write data to a file at current position
Seek	Reposition the file pointer
Close	Close an open file
Delete	Delete a file and free its space
Truncate	Truncate file to a specified size

File Access Methods

1. Sequential Access

Data is read/written in order from beginning. File pointer moves forward automatically. Best for tape drives.

SEQUENTIAL ACCESS	
Start -> Record 1 -> Record 2 -> Record 3 -> ... -> Record N	
File pointer moves forward only	
Simple to implement	
Used for: text files, log files, tape storage	

2. Direct (Random) Access

Can read/write any block directly. File pointer can move to any position. Best for disk storage.

+-----+	
	DIRECT ACCESS
+-----+	
	File blocks: [0] [1] [2] [3] [4] [5] ... [N]
	Can access any block directly (e.g., block 4)
	File pointer can move anywhere
	Used for: databases, disk storage
+-----+	

3. Indexed Access

Uses an index table to locate records. The index contains pointers to record locations. Very fast random access.

+-----+		
	INDEXED ACCESS	
+-----+		
	Index Table:	
	+-----+	
	Key	Block
+-----+		

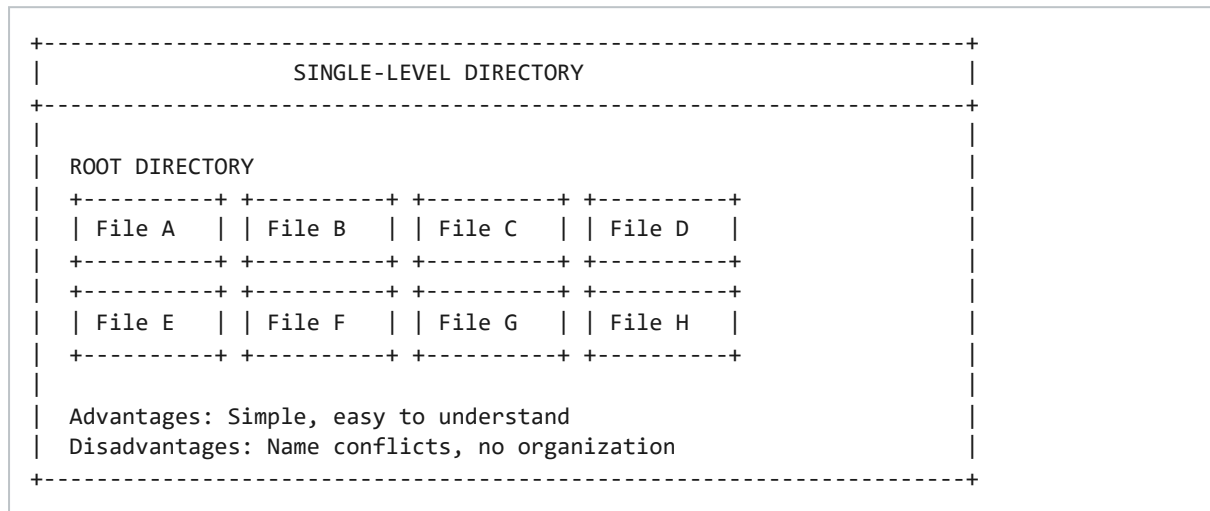
+-----+-----+	
ABC 5	
DEF 2	
GHI 8	
+-----+-----+	
Used for: database indexes, file systems	
+-----+-----+	

Comparison

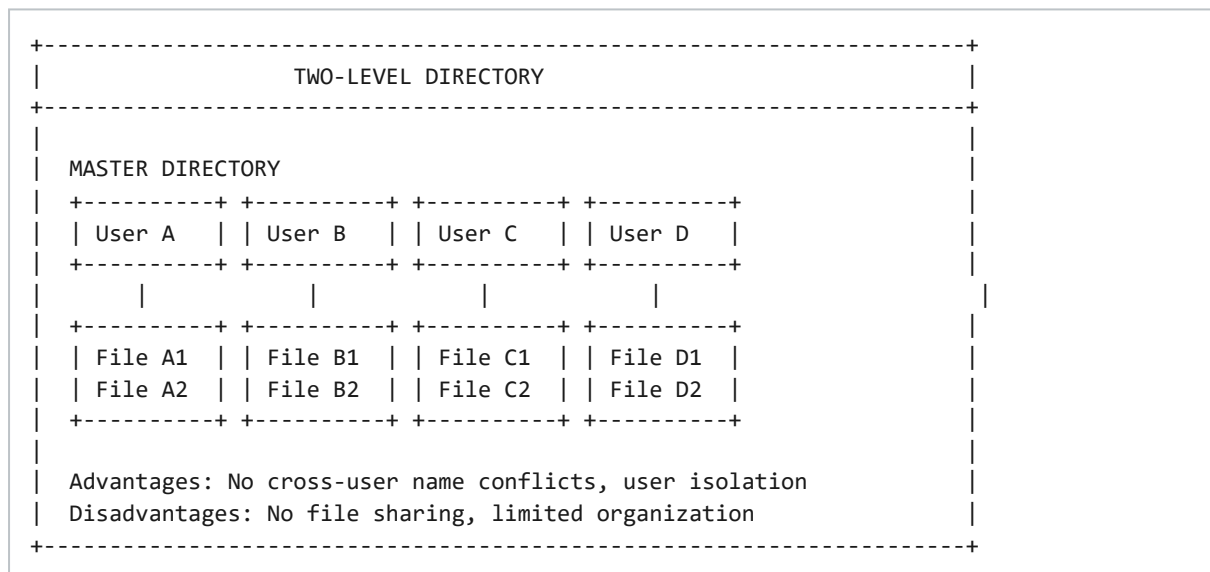
Aspect	Sequential	Direct	Indexed
Access Order	Linear	Any position	Via index
Speed (random)	Slow	Fast	Very fast
Storage Overhead	Minimal	Minimal	Index space
Best For	Tape, text files	Disk, databases	Large databases
Complexity	Simple	Moderate	Complex

Directory Structure

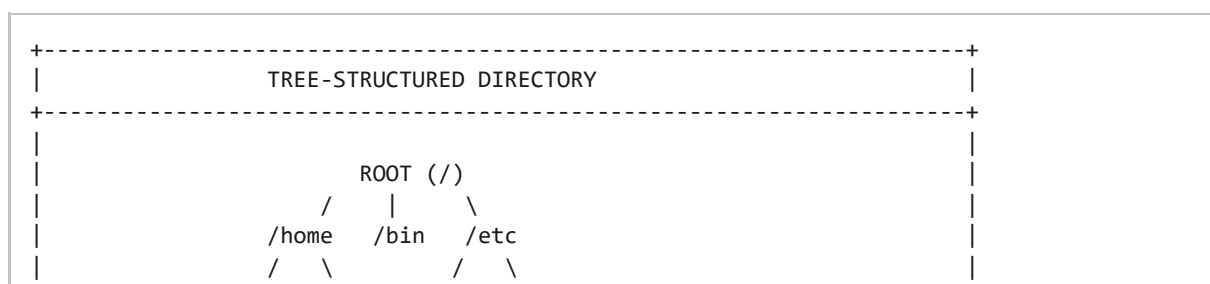
1. Single-Level Directory

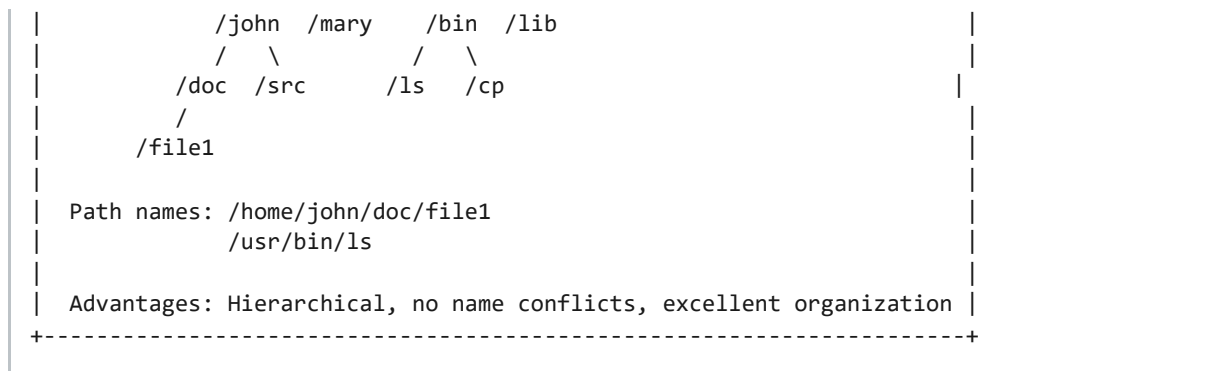


2. Two-Level Directory



3. Tree-Structured Directory





4. Acyclic-Graph Directory

Allows directories to have shared subdirectories and files (via links). Forms a directed acyclic graph. Enables file sharing but requires garbage collection.

5. General-Graph Directory

Allows cycles in the directory structure. More flexible but requires cycle detection to avoid infinite traversal.

Directory Structure Comparison

Feature	Single-Level	Two-Level	Tree	Acyclic Graph
Depth	1	2	Unlimited	Unlimited
Name Conflicts	Yes	No (between users)	No	No
Organization	Poor	Good	Excellent	Excellent
File Sharing	Easy	Difficult	Through links	Built-in
Complexity	Simple	Medium	Complex	Very complex

File Protection

File Protection ensures files are accessed only by authorized users and only in authorized ways.

Protection Methods

1. Access Control Lists (ACL)

User	Read	Write	Execute
Owner	Yes	Yes	Yes
Group	Yes	No	Yes
Others	No	No	No

2. UNIX Permission Model

```
Example: -rw-r--r--
          |  |  |
          |  |  +--- Others: Read only
          |  +----- Group: Read only
          +----- Owner: Read and Write
          |
          +--- File type (- = file, d = directory)
```

Permission	File	Directory
Read (r)	View file contents	List directory contents
Write (w)	Modify file contents	Create/delete files
Execute (x)	Run as program	Access directory

3. Password Protection

File encrypted with password. Password required to access file. Strong protection but risk of password loss.

4. Encryption

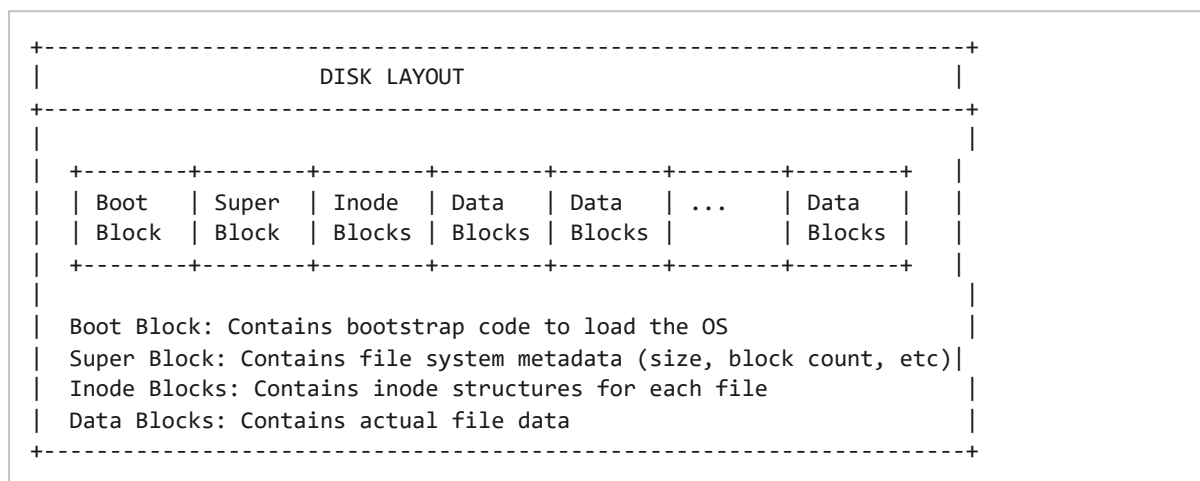
File data is encrypted and stored. Decryption key required for access. Strongest protection for sensitive data.

Protection in Different Systems

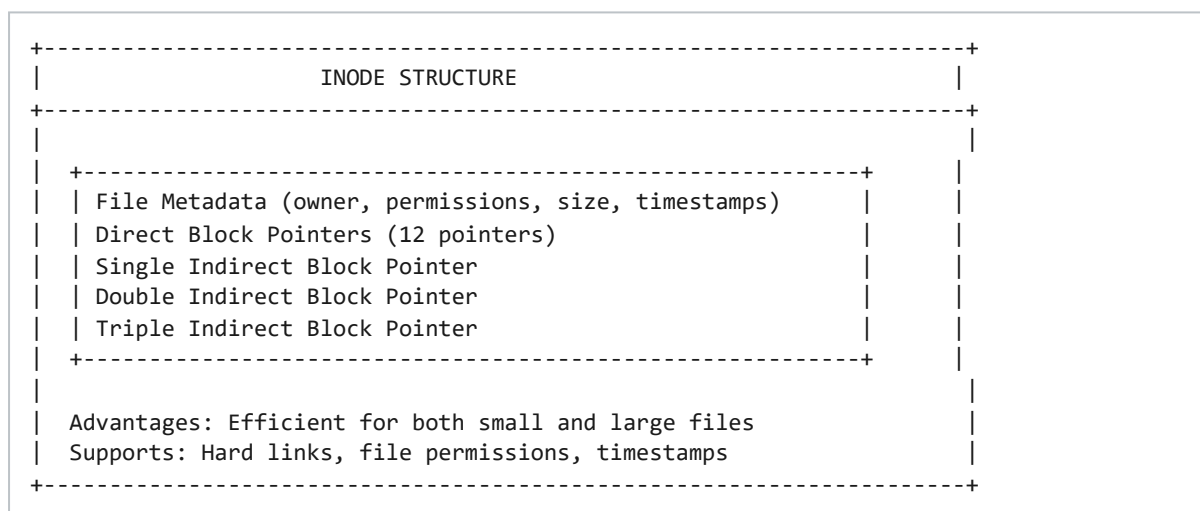
System	Protection Method	Example
UNIX/Linux	User/Group/Others with rwx	<code>chmod 755 filename</code>
Windows	ACL (Access Control Lists)	User permissions, Groups
Network	Authentication + Authorization	Login, Password, Roles

File System Implementation

Disk Layout



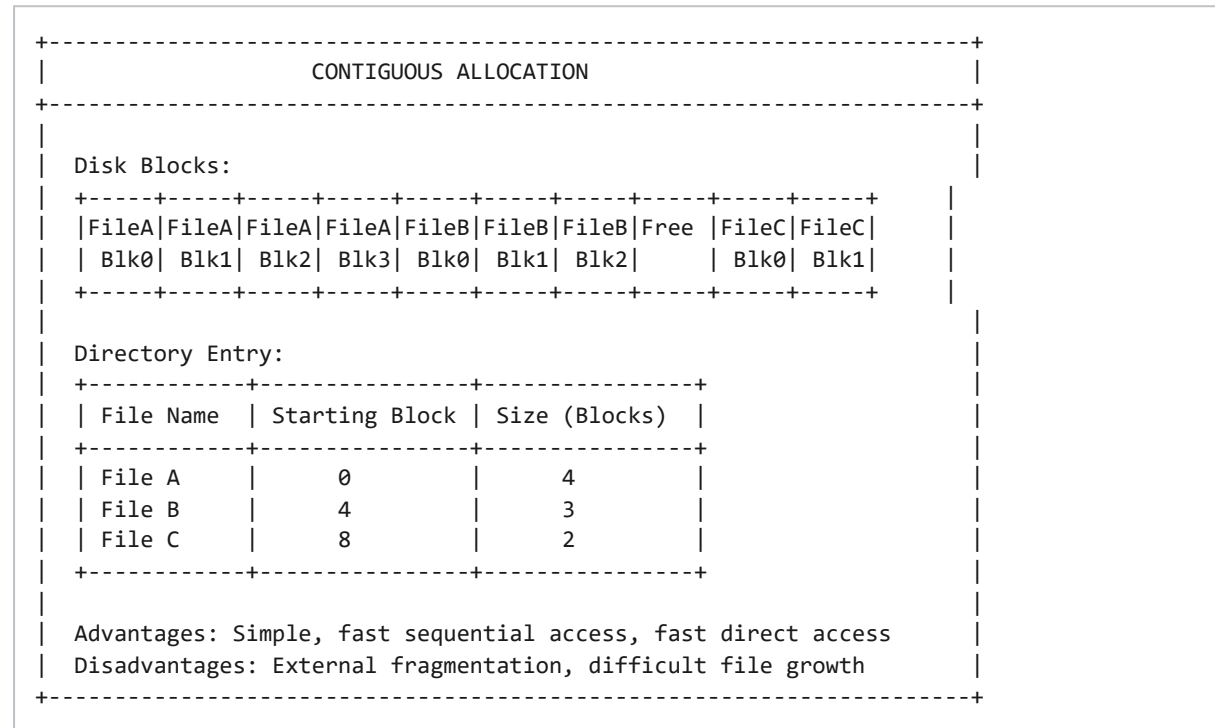
Inode Structure



Allocation Methods

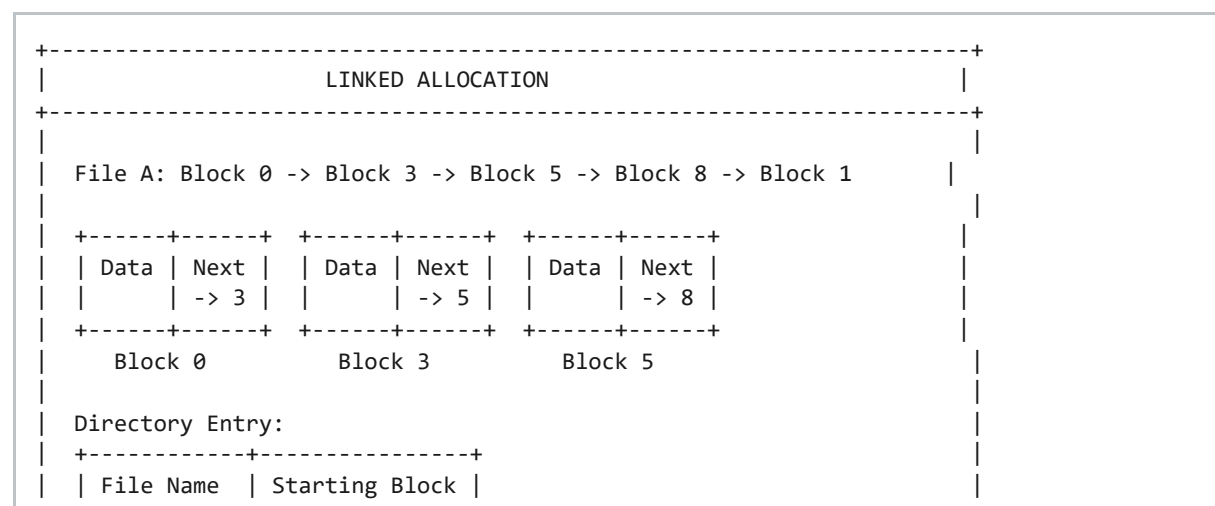
1. Contiguous Allocation

Each file occupies a **single contiguous block** of disk space. All blocks are located consecutively.



2. Linked Allocation

Each file is a **linked list** of disk blocks. Each block contains a pointer to the next block.



```

+-----+-----+
| File A |      0      |
+-----+-----+

```

Advantages: No external fragmentation, easy file growth

Disadvantages: Slow sequential access, no direct access, ptr waste

File Allocation Table (FAT)

FILE ALLOCATION TABLE (FAT)

```

+-----+-----+ +-----+-----+ +-----+-----+
| Block | Next | | Block | Next | | Block | Next |
+-----+-----+ +-----+-----+ +-----+-----+
|  0    |  3   | |  4    |  2   | |  8    |  1   |
|  1    | Null | |  5    |  8   | |  9    | Null |
|  2    |  6   | |  6    | Null | | ...   | ...  |
|  3    |  5   | |  7    |  4   | | ...   | ...  |
+-----+-----+ +-----+-----+ +-----+-----+

```

FAT is stored in memory during operation for fast access

3. Indexed Allocation

All pointers to a file's blocks are stored in a separate **index block**.

INDEXED ALLOCATION

Index Block for File A: [0, 3, 5, 8, 1, ...]

```

+-----+-----+-----+-----+-----+-----+-----+
| Index Block | 0 | 3 | 5 | 8 | 1 | ... | ... |
+-----+-----+-----+-----+-----+-----+-----+

```

```

      |   |   |   |   |
      v   v   v   v   v

```

```

+-----+-----+-----+-----+-----+
| B0 | B3 | B5 | B8 | B1 | (Data Blocks)
+-----+-----+-----+-----+-----+

```

Directory Entry:

```

+-----+-----+
| File Name | Index Block |
+-----+-----+
| File A    | Index Block 10 |
+-----+-----+

```

Advantages: Direct access, no external fragmentation, easy growth

Disadvantages: Space for index block, overhead for small files

Indexed Allocation Schemes

Scheme	Description	Advantages	Disadvantages
Single-Level Index	One index block	Simple, Fast	Limited file size
Multi-Level Index	Index blocks point to more index blocks	Large files supported	Slower access
Combined Scheme	Direct + single/triple indirect	Efficient for all sizes	Complex

Allocation Methods Comparison

Aspect	Contiguous	Linked	Indexed
Sequential Access	Excellent	Good	Good
Direct Access	Excellent	Poor	Good
External Fragmentation	Yes	No	No
File Growth	Difficult	Easy	Easy
Space Overhead	Minimal	Extra for pointers	Extra for index
Reliability	Good	Bad	Good
Complexity	Simple	Medium	Complex

Free Space Management

Tracks which blocks on disk are free and which are used.

1. Bit Vector (Bit Map)

Each block represented by one bit. 1 = free, 0 = used.

Example: Bits: [1, 1, 0, 1, 0, 1, 1, 0]

Free blocks: 0, 1, 3, 5, 6

```
+-----+
| Bitmap: | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 | |
+-----+
| Block:  | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |10 |11 | |
+-----+
```

2. Free List (Linked List)

Free blocks are linked together. Each free block points to the next free block. Simple but slow traversal.

Free Block List: Block 2 -> Block 7 -> Block 9 -> Block 15 -> ...

3. Grouping

The first free block stores a list of addresses of other free blocks. Reduces the overhead of linked list traversal.

4. Counting

Store (block address, count) pairs. Good for contiguous free blocks. Example: (Block 10, 5) means 5 free blocks starting from block 10.

Technique	Space Overhead	Speed	Best For
Bit Vector	Large (unused if many blocks used)	Fast	Small to medium disks
Free List	Minimal	Slow	Simple systems

Grouping	Low	Moderate	Large disks
Counting	Low	Fast	Contiguous free blocks

Disk Scheduling

Disk scheduling algorithms optimize the order of disk I/O requests to minimize **seek time** (the time for the disk arm to move to the correct track).

1. FCFS (First-Come-First-Served)

Requests are serviced in the order they arrive. Simple but can cause long seek times.

```
Request Queue: 98, 183, 37, 122, 14, 124, 65, 67
Initial Head Position: 53

Head Movement: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total Seek: |98-53| + |183-98| + |37-183| + |122-37| + |14-122| + |124-14| + |65-124| + |67-124|
           = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640 cylinders
```



2. SSTF (Shortest Seek Time First)

Service the request with the shortest seek time from the current position. May cause starvation.

```
Request Queue: 98, 183, 37, 122, 14, 124, 65, 67
Initial Head: 53

Order: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total Seek: 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = 236 cylinders
```

3. SCAN (Elevator Algorithm)

The disk arm moves in one direction, servicing all requests in its path, then reverses direction.

```
Request Queue: 98, 183, 37, 122, 14, 124, 65, 67
Initial Head: 53 (moving towards 0)

Order: 53 -> 37 -> 14 -> 0 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183
Total Seek: 16 + 23 + 14 + 65 + 2 + 31 + 24 + 2 + 59 = 236 cylinders
```

4. C-SCAN (Circular SCAN)

Disk arm moves in one direction, servicing requests on the way. When it reaches the end, it jumps to the other end without servicing any requests on the return.

Request Queue: 98, 183, 37, 122, 14, 124, 65, 67

Initial Head: 53 (moving towards 199)

Order: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 -> 14 -> 37

Total Seek: $12 + 2 + 31 + 24 + 2 + 59 + 16 + 199 + 14 + 23 = 382$ cylinders

5. LOOK

Similar to SCAN but only goes as far as the last request in each direction (no need to go to the end).

6. C-LOOK

Similar to C-SCAN but only goes to the last request in each direction before reversing.

Disk Scheduling Comparison

Algorithm	Throughput	Starvation	Fairness	Complexity
FCFS	Low	No	High	Simple
SSTF	High	Yes	Low	Moderate
SCAN	High	No	Medium	Moderate
C-SCAN	High	No	High	Moderate
LOOK	High	No	Medium	Moderate
C-LOOK	Highest	No	High	Moderate

Disk Management

Partitioning

Dividing a physical disk into multiple logical units (partitions). Each partition appears as a separate disk to the OS. Can boot different OS from different partitions.

Formatting

- **Low-level Formatting:** Divides disk into sectors that can be read/written by the disk controller
- **High-level Formatting:** Creates file system structures (boot block, super block, inodes, etc.) on a partition

Bad Block Management

Handles defective blocks on the disk surface:

- **Sector Sparing:** Bad sector replaced with a spare sector
- **Sector Sliding:** All sectors shifted to skip bad sectors
- **Error-Correcting Code (ECC):** Detects and corrects errors in data

I/O Management

I/O Hardware

I/O devices communicate with the CPU via **ports**, **buses**, and **device controllers**.

Polling

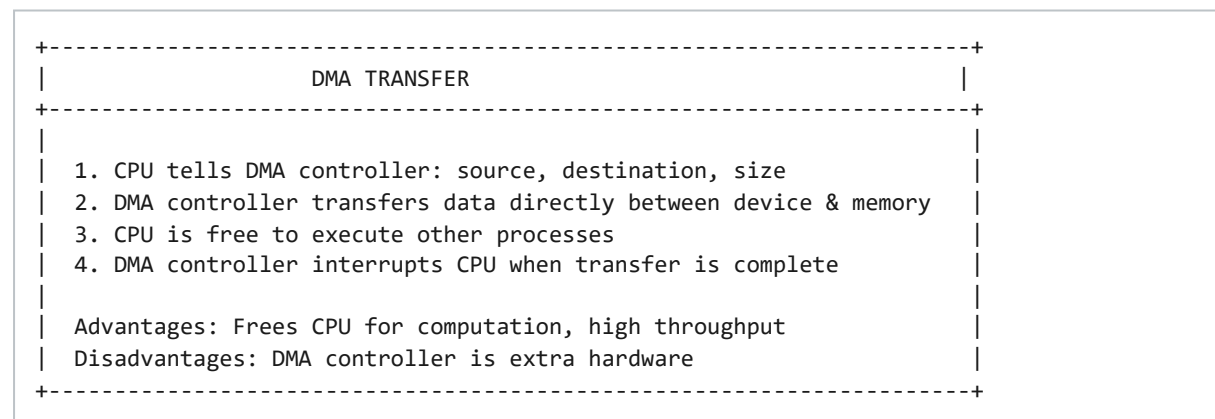
CPU continuously checks the status register of the device until it is ready. Simple but wastes CPU cycles.

Interrupts

Device sends an interrupt signal to CPU when it is ready. CPU suspends current process and handles the I/O. Much more efficient than polling.

DMA (Direct Memory Access)

DMA controller transfers data directly between I/O device and memory without CPU involvement. CPU is only involved at the start and end of transfer. Frees CPU for other tasks.



I/O Software Layers

Layer	Function
User-Level I/O	System calls (read, write, open, close)

Device-Independent OS Software	Buffering, caching, naming, protection
Device Drivers	Communicate with device controller
Interrupt Handlers	Handle interrupts from devices
Hardware	Device controller, device

Buffering

Buffers are temporary storage areas used to transfer data between devices with different speeds.

- **Single Buffering:** One buffer used
- **Double Buffering:** Two alternating buffers
- **Circular Buffering:** Multiple buffers in a circle

Spooling

Spooling (Simultaneous Peripheral Operations On-Line) manages jobs for dedicated I/O devices (like printers). Jobs are stored on disk and sent to the device one at a time.

RAID Levels

RAID (Redundant Array of Independent Disks) combines multiple physical disks into a single logical unit for performance and/or reliability.

RAID Level	Description	Min Disks	Fault Tolerance	Use Case
RAID 0	Striping - data split across disks; high performance, no redundancy	2	None	High performance, non-critical data
RAID 1	Mirroring - identical copies on two disks; high reliability	2	1 disk failure	Critical data, OS
RAID 2	Bit-level striping with Hamming code ECC	3	1 disk failure	Obsolete (used in early systems)
RAID 3	Byte-level striping with dedicated parity disk	3	1 disk failure	Large sequential data
RAID 4	Block-level striping with dedicated parity disk	3	1 disk failure	Small read requests
RAID 5	Block-level striping with distributed parity	3	1 disk failure	Most common (good balance)
RAID 6	Block-level striping with dual distributed parity	4	2 disk failures	High reliability required

Nested RAID

Level	Description	Min Disks
RAID 10 (1+0)	Mirrored stripes - combines RAID 1 and RAID 0	4
RAID 01 (0+1)	Striped mirrors - combines RAID 0 and RAID 1	4

RAID 50 (5+0)	Striped parity - combines RAID 5 and RAID 0	6
---------------	---	---

Important Formulas

Concept	Formula
Seek Time (FCFS)	$\sum Request_i - Request_{i-1} $
Rotational Latency (avg)	Time for half rotation = $1 / (2 \times RPM / 60)$
Access Time	Seek Time + Rotational Latency + Transfer Time
Transfer Rate	Data transferred / Time taken
RAID Capacity (R0)	$N \times \text{Disk Capacity}$ (N = number of disks)
RAID Capacity (R1)	Disk Capacity (with N=2)
RAID Capacity (R5)	$(N - 1) \times \text{Disk Capacity}$

Quick Revision Points

Topic	Key Points
File Types	Data, Program, System, Special files
File Operations	Create, Open, Read, Write, Seek, Close, Delete
Access Methods	Sequential, Direct, Indexed
Allocation Methods	Contiguous, Linked, Indexed
Free Space Management	Bit Vector, Linked List, Grouping, Counting
Directory Structures	Single-Level, Two-Level, Tree-Structured, Acyclic-Graph
File Protection	ACL, Permissions, Password, Encryption
Disk Scheduling	FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK

RAID Levels	0-Striping, 1-Mirroring, 5-Parity, 6-Dual Parity
-------------	--

Questions

SECTION A: Very Short Answer Questions (2-3 Marks)

Q1. What is a File? What are the different types of files?

Answer: A File is a named collection of related information on secondary storage.

Types: 1. Data Files (text, images, audio, video), 2. Program Files (executables, DLLs, source), 3. System Files (OS files, config, logs), 4. Special Files (device, pipe, socket).

Q2. List any five operations on files.

Answer: Create, Open, Read, Write, Close, Seek, Delete, Truncate.

Q3. Difference between Sequential and Direct Access?

Sequential Access	Direct Access
Must read records in order	Can read any record directly
File pointer moves forward	File pointer moves anywhere
Slow for random access	Fast for random access
Best for tape	Best for disk

Q4. What is Free Space Management? Name its techniques.

Answer: Free Space Management tracks which blocks on disk are free. **Techniques:** Bit Vector, Linked List, Grouping, Counting.

Q5. What are the three allocation methods in file systems?

Answer: 1. Contiguous Allocation, 2. Linked Allocation, 3. Indexed Allocation.

Q6. Define File Protection. What are the common protection methods?

Answer: File Protection ensures files are accessed only by authorized users.

Methods: ACL, User/Group/Others permissions, Password protection, Encryption.

Q7. What is Internal Fragmentation (in file systems)?

Answer: Occurs when a file does not fill the last block completely. E.g., block size = 4KB, file = 3KB → 1KB wasted.

Q8. What is a Directory? Name its types.

Answer: A Directory organizes files. **Types:** Single-Level, Two-Level, Tree-Structured, Acyclic-Graph, General-Graph.

Q9. What is FAT?

Answer: FAT (File Allocation Table) is a linked allocation scheme where a table in memory stores next-block pointers. Used in FAT16/FAT32 file systems.

Q10. Advantages of Indexed Allocation over Linked Allocation?

Aspect	Indexed	Linked
Direct Access	Supported	Not supported
Speed	Fast	Slow
Reliability	Good	Bad
Space Overhead	Higher	Lower

SECTION B: Short Answer Questions (5 Marks)

Q11. Explain the different types of Access Methods in detail.

Answer: Sequential Access: Data read/written in order from beginning; best for tape. **Direct Access:** Can read/write any block directly; best for disk. **Indexed Access:** Uses index table to locate records; very fast random access with extra storage overhead.

Q12. Explain Contiguous, Linked, and Indexed Allocation methods with diagrams.

Answer: Contiguous: File occupies consecutive blocks; fast but external fragmentation. **Linked:** File blocks linked by pointers; no external fragmentation but no direct access. **Indexed:** Index block stores all pointers; supports direct access with index overhead.

Q13. Explain the different Directory Structures.

Answer: Single-Level: All files in one directory; simple but name conflicts. **Two-Level:** Master with user subdirectories; no cross-user conflicts. **Tree-Structured:** Hierarchical with unlimited depth; most common. **Acyclic-Graph:** Allows sharing via links; requires garbage collection.

Q14. Explain Free Space Management techniques in detail.

Answer: Bit Vector: Each block = 1 bit. **Free List:** Free blocks linked together. **Grouping:** First free block stores list of free blocks. **Counting:** Store (block address, count) pairs for contiguous free blocks.

Q15. Explain File Protection mechanisms.

Answer: ACL: Per-user permission list. **UNIX Permissions:** Owner/Group/Others with rwx. **Password:** File encrypted with password. **Encryption:** Data stored encrypted with decryption key required.

SECTION C: Long Answer Questions (10-15 Marks)**Q16. Explain all file allocation methods with advantages, disadvantages, and diagrams.**

Answer: Contiguous: Simple, fast, but external fragmentation and difficult growth. **Linked:** No external fragmentation, easy growth, but no direct access and pointer overhead. **Indexed:** Direct access, no fragmentation, but index overhead and complex.

Q17. Explain the Tree-Structured Directory System with examples.

Answer: Hierarchical structure with unlimited depth. Uses path names (absolute/relative). Supports file and directory operations. Most common modern directory structure (Unix, Windows). Example: `/home/user/doc/file.txt`.

Q18. Write detailed notes on: (a) File Access Methods (b) Directory Structures (c) File Protection

Answer: (a) Sequential, Direct, Indexed access methods. (b) Single-Level, Two-Level, Tree-Structured, Acyclic Graph directories. (c) ACL, User/Group/Others permissions, Password Protection, Encryption.

End of UNIT - IV: File Management